To-Do Task API

Stage 1 Tutorial: raw PHP + MySQL behind NGINX

Josh Cramer, CSC 640, HW4

What this API does

Tasks are stored in categories (Work, Personal, School).

- **Anyone** can read categories and tasks.
- **Registered, logged-in users** can create, update and delete tasks.
- Logging in returns a **bearer token**; the three write endpoints reject a missing or invalid token with **401**.

Stage 1 is the baseline: plain PHP + MySQL (PDO), served through NGINX. Stage 2 rebuilds the same API in Laravel and runs the **same acceptance tests**.

How a request travels

```
curl / browser
  -> NGINX (port 8080)     also serves test.html
  -> PHP-FPM               runs the PHP code
  -> index.php             routes the request, builds the JSON
  -> MySQL (todo_api)      users, api_tokens, categories, tasks
```

- `index.php` is the single entry point (front controller): it reads the method and path, then calls one handler function.
- `database.php` opens the MySQL connection with PDO and prepared statements.

Conventions

- Base URL: `http://localhost:8080`, all paths start with `/api`.
- Requests with a body send JSON and `Content-Type: application/json`.
- Every response is JSON in one of two shapes:

```
{"success": true,  "data": ...}
{"success": false, "error": "what went wrong"}
```

Status	Meaning
200 / 201	OK / Created
400	Bad input (missing field, invalid value, non-numeric id)
401	Missing or invalid bearer token, or wrong login
404	Unknown resource or id
405	Method not allowed (response has an `Allow` header)
409	Email already registered

The 10 endpoints

#	Method and path	Auth
1	`POST /api/register`	public
2	`POST /api/login`	public
3	`GET /api/categories`	public
4	`GET /api/categories/{id}`	public
5	`GET /api/categories/{id}/tasks`	public
6	`GET /api/tasks`	public
7	`GET /api/tasks/{id}`	public
8	`POST /api/tasks`	**bearer token**
9	`PUT /api/tasks/{id}`	**bearer token**
10	`DELETE /api/tasks/{id}`	**bearer token**

Also: `GET /api` returns a short list of the endpoints.

Authentication: the bearer token flow

1. **Register** once, then **log in** with email and password.
2. The login response contains a `token` (64 hex characters).
3. Send it on every protected request:

```
TOKEN="paste-the-token-here"
curl -H "Authorization: Bearer $TOKEN" ...
```

4. The server hashes the token (SHA-256), looks the hash up in `api_tokens`, and finds the user. No match means **401**.

Security choices: passwords are stored as bcrypt hashes; tokens are stored only as SHA-256 hashes, so a database leak does not reveal usable tokens. Authentication is checked **before** the request body is validated.

Data model (MySQL)

```
users(id, name, email UNIQUE, password_hash, created_at)
api_tokens(id, user_id -> users, token_hash UNIQUE, created_at)
categories(id, name UNIQUE, created_at)
tasks(id, title, done, category_id -> categories, user_id -> users,
      created_at, updated_at)
```

- Full schema: `code/stage1/api/schema.sql`. Seed data: `seed.sql` (categories 1 to 3, tasks 1 and 2, user `seed@example.com`).
- A task belongs to one category and was created by one user.
- `done` is stored as 0/1 and returned as JSON `true` / `false`.

1. POST /api/register (public)

Input (JSON): `name` (1-100 chars), `email` (valid), `password` (6+ chars)
Output: `201` with the new user (never the password)

```
curl -X POST http://localhost:8080/api/register \
  -H "Content-Type: application/json" \
  -d '{"name":"Ada Lovelace","email":"ada@example.com","password":"secret123"}'
```

```
201 {"success":true,"message":"User registered",
     "data":{"id":2,"name":"Ada Lovelace","email":"ada@example.com"}}
```

Failures: same email again gives `409 {"success":false,"error":"Email is already registered"}`; invalid email gives `400 {"success":false,"error":"email is not a valid email address"}`; missing fields or a short password give `400`.

2. POST /api/login (public)

Input (JSON): `email`, `password`
Output: `200` with a `token` and the user

```
curl -X POST http://localhost:8080/api/login \
  -H "Content-Type: application/json" \
  -d '{"email":"ada@example.com","password":"secret123"}'
```

```
200 {"success":true,"token":"e687a779...a568a4",
     "data":{"id":2,"name":"Ada Lovelace","email":"ada@example.com"}}
```

Failures: wrong password gives `401 {"success":false,"error":"Invalid credentials"}` and **no** token; missing fields give `400`.

(The token above is shortened; a real one is 64 characters.)

3-4. GET /api/categories and /api/categories/{id} (public)

Input: none (the `{id}` is a positive integer). **Output:** `200`

```
curl http://localhost:8080/api/categories
```

```
200 {"success":true,"data":[{"id":1,"name":"Work","created_at":"..."},
     {"id":2,"name":"Personal","created_at":"..."},
     {"id":3,"name":"School","created_at":"..."}],"count":3}
```

```
curl http://localhost:8080/api/categories/1
# 200 {"success":true,"data":{"id":1,"name":"Work","created_at":"..."}}
curl http://localhost:8080/api/categories/999999
# 404 {"success":false,"error":"Category not found"}
curl http://localhost:8080/api/categories/abc
# 400 {"success":false,"error":"Invalid id"}
```

5. GET /api/categories/{id}/tasks (public)

Input: category id. **Output:** `200` with that category's tasks (possibly an empty list) and a `count`

```
curl http://localhost:8080/api/categories/3/tasks
```

```
200 {"success":true,"data":[{"id":1,"title":"Finish HW4 plan",
     "done":false,"category_id":3,"user_id":1,
     "created_at":"...","updated_at":"..."}],"count":1}
```

Failures: unknown category `404 {"success":false,"error":"Category not found"}`; non-numeric id `400 {"success":false,"error":"Invalid id"}`.

6-7. GET /api/tasks and /api/tasks/{id} (public)

Input: none. **Output:** `200`. The list also has a `count`.

```
curl http://localhost:8080/api/tasks          # all tasks
curl http://localhost:8080/api/tasks/1        # one task
```

```
200 {"success":true,"data":{"id":1,"title":"Finish HW4 plan","done":false,
     "category_id":3,"user_id":1,"created_at":"...","updated_at":"..."}}
```

Failures:

`GET /api/tasks/999999` gives `404 {"success":false,"error":"Task not found"}`

`GET /api/tasks/abc` gives `400 {"success":false,"error":"Invalid id"}`

8. POST /api/tasks (bearer token)

Input (JSON): `title` (1-200 chars), `category_id` (existing category), optional `done` (true/false, default false)
Output: `201` with the created task, owned by the logged-in user

```
curl -X POST http://localhost:8080/api/tasks \
  -H "Content-Type: application/json" -H "Authorization: Bearer $TOKEN" \
  -d '{"title":"Write tutorial","category_id":3}'
```

```
201 {"success":true,"message":"Task created","data":{"id":3,
     "title":"Write tutorial","done":false,"category_id":3,"user_id":2,...}}
```

8. POST /api/tasks: failures

Request	Result
no `Authorization` header	`401 {"success":false,"error":"Missing bearer token"}`
`Authorization: Bearer invalid-token-123`	`401 {"success":false,"error":"Invalid token"}`
body `{"category_id":3}` (no title)	`400 {"success":false,"error":"title is required"}`
`"category_id": 999999`	`400` (`category_id does not exist`)

The token check runs first, so a missing token gives 401 even when the body is also wrong.

9. PUT /api/tasks/{id} (bearer token)

Input (JSON): a **full replacement**: `title`, `category_id` and `done` are all required

Output: `200` with the updated task

```
curl -X PUT http://localhost:8080/api/tasks/3 \
  -H "Content-Type: application/json" -H "Authorization: Bearer $TOKEN" \
  -d '{"title":"Write tutorial (done)","category_id":3,"done":true}'
```

```
200 {"success":true,"message":"Task updated","data":{"id":3,
     "title":"Write tutorial (done)","done":true,"category_id":3,...}}
```

Failures: no token or invalid token `401`; `{"title":"Only a title"}` gives `400 {"success":false,"error":"category_id is required"}`; unknown id `404 {"success":false,"error":"Task not found"}`.

10. DELETE /api/tasks/{id} (bearer token)

Input: none. **Output:** `200` with the task that was deleted

```
curl -X DELETE http://localhost:8080/api/tasks/3 \
  -H "Authorization: Bearer $TOKEN"
```

```
200 {"success":true,"message":"Task deleted","data":{"id":3,...}}
```

Failures:

- no or invalid token: `401`
- deleting the same id again: `404 {"success":false,"error":"Task not found"}`
- after a delete, `GET /api/tasks/3` also returns `404`

Other errors every endpoint can return

Request	Response
`GET /api/unknown`	`404 {"success":false,"error":"Resource not found"}`
`PATCH /api/tasks/1`	`405 {"success":false,"error":"Method not allowed"}` plus an `Allow: GET, PUT, DELETE` header
malformed or non-object JSON body	`400 {"success":false,"error":"Request body must be a valid JSON object"}`
database failure	`500 {"success":false,"error":"Database error"}` (details go to the server log, not the client)

Setup and run (Ubuntu / WSL2)

Needs `nginx`, `php-fpm`, `php-mysql`, `mysql-server`, `curl`.

```
bash code/stage1/setup.sh      # database, user, schema, seed data, NGINX site on 8080
bash code/stage1/run.sh        # start services later and check the API answers
curl http://localhost:8080/api # should print the endpoint list as JSON
```

Common problem: NGINX will not start if another program already uses port 80. Disable NGINX's default site (`sudo rm -f /etc/nginx/sites-enabled/default`), then `sudo systemctl start nginx`. The API itself only uses port 8080.

Testing

1. Executable acceptance tests (success and failure for every endpoint, token cases included). The script exits nonzero if any assertion fails and saves its output.

```
bash code/stage1/reset_db.sh                    # known seed data
STAGE=stage1 bash code/tests/run_tests.sh       # result: 52 passed, 0 failed
```

Cases and IDs: `acceptance.md`. Saved output: `code/tests/results/stage1_results.txt`.

2. HTML/JavaScript test page: open `http://localhost:8080/test.html`, click **Send** on the login row first (it stores the token), then try each endpoint. Success is shown in green and errors in red.

Known limitations (Stage 1)

- **No ownership check:** any logged-in user can update or delete **any** task, not only their own.
- **Tokens never expire** and there is no logout endpoint.
- No pagination, filtering or rate limiting.
- Passwords need only 6 characters.
- Responses are tested for status and key fields, not for every field of every object.

These are known gaps in the Stage 1 baseline, kept as written so Stage 2 can be compared fairly.

Credits and AI assistance

- Reused and adapted from ASE 230 (`nkuase/ase230`): module 1 `6_RestAPI/api/` (front controller, JSON responses, PDO pattern, curl test style), `5_Webserver_using_NGINX/nginx.conf`, and `7_Security/Basics/bearer/`.
- AI assistance: Claude (Anthropic) helped write the Stage 1 code, the acceptance test script and the drafts of these documents. I ran everything on my own machine (WSL2 Ubuntu, MySQL, NGINX), and the test results above are from that run.
- Source and results: https://github.com/Jcramer777/Josh-Cramer-Repo-CSC640